

OdoGo

An intra-campus ride sharing platform

CS253 Project by the team **Thank God I Forked**

Mentor TA: **Seetaramayya Lavu**

Github Repo: github.com/Chwiz67/odogo_app

Aditya Sai Vemparala 240068

Aiklavayaveer Singh Rana 240075

Archit Agarwal 240171

Arish Maurya 240177

Arman Singhal 240184

Arnav Gupta 240189

Avtansh Gargya 240234

Inesh Aggarwal 240465

Karthic N A 240524

Shashwath Ram 240966



ODOGO



- This application aims to be a **one-stop solution** to streamline campus transportation and logistics within IITK.
- Problems faced with current campus transportation:
 - Unpredictable e-rickshaw availability
 - Need to physically walk to rickshaw stands
 - Relying on verbal negotiation for availability
 - Inefficient, ad-hoc transit processes
- Potential Users: **IITK students and staff** needing **campus rides** + **E-rickshaw operators** looking to optimise their workflow
- Helps in connecting: Commuters with available e-rickshaws
- Key Features & Constraints:
 - Fully **autonomous**, peer-to-peer marketplace (No central dispatcher)
 - **Automatic ride matching** and conflict resolution. '**First-Accept**' egalitarian driver assignment
 - Scheduled **advance bookings**
 - Integrates GPS, **OpenStreetMap**, and secure OAuth gateways

REQUIREMENTS

Connecting Commuters and Transport Providers

- The mobile application must provide commuters the functionality to request immediate rides via a "**Book Now**" feature, displaying an **estimated time of arrival (ETA)**.
- It should also offer a "**Schedule Booking**" option, allowing users to proactively plan travel by booking rides 20 minutes in advance.
- The app must provide a **live map interface** for the commuter to visualise the real-time location.
- To streamline the booking process, the software should allow commuters to save favourite destinations and select from a **static list of standard campus locations**.
- The platform must provide a **messaging and native calling interface** to the commuters and the drivers so that they can directly contact the assigned drivers and commuters respectively.
- It should allow commuters to manage their trips by offering a **cancellation feature**, constrained to a maximum of two cancellations per 15 minutes to prevent system abuse.
- The app must include a **profile customisation** section where users can input specific details like their phone number, address etc which helps drivers navigate efficiently to pickup points.

REQUIREMENTS

Equipping the E-Rickshaw Fleet

- The driver's interface must be specifically designed for the driving context, featuring **high-contrast visuals, thumb-friendly buttons** to minimise interaction requirements while the vehicle is in motion.
- The software should provide an intuitive, distraction-free dashboard for drivers to easily toggle their "**Online**" or "**Offline**" status, including an Auto Offline feature that manages availability depending on the distance from the campus.
- The software should employ a "**First-Accept**" dispatch algorithm, utilising database transactions to prevent double-booking and allow any available driver to claim a broadcasted job equitably.
- The application must include a dedicated section for drivers to view, manage, and access their upcoming **pre-booked scheduled trips**.
- **Upon accepting a ride, the application must immediately provide the driver with the passenger's details, including their name, phone number and specific address.**
- The software must implement a **two-step ride completion process**, allowing drivers to initiate an "**End Trip**" function upon arrival, which then requires a **mandatory confirmation** from the commuter before the ride is officially ended.

IMPLEMENTATION

Frontend was built using Dart as the programming language. We used Flutter as the framework for both the Commuter and Driver interfaces, and flutter_riverpod was used for state management across the application.

Backend was written in Dart and Firebase was used as the serverless backend, eliminating the need for manual server provisioning.

Cloud Firestore was used to handle our primary **Database**, storing user profiles, ride queues and scheduled rides in a flexible, document-based schema. Firebase Realtime Database was used for storing live driver GPS coordinates due to its low-latency, high-frequency write capabilities.





DEMO TIME!!!!

LESSONS LEARNT



Working in a large team taught us that communication gaps are the biggest bugs.

We learned to give and receive feedback without taking it personally.

Dividing work across 10 people taught us the art of delegation.

We learned that documentation is necessary and part of the product.

Sitting together and debugging taught us the power of pair problem-solving.

Managing conflicting opinions taught us how to reach consensus under pressure.

FUTURE DEV PLANS

Integrated Digital Payment Gateway will be introduced in future versions with UPI integration, allowing seamless in-app payments while currently the platform acts as a zero-payment intermediary.

Emergency SOS Integration will be added as a dedicated module that directly pings the campus Security and Intelligence Services (SIS) dashboard with the vehicle's real-time coordinates in the event of distress.

Demand Heatmap will overlay real-time high-footfall zones on the driver's map, guiding e-rickshaw operators to where they are needed most, such as the Academic Area during lunch or OAT during festivals.

Native iOS Support will compile the existing Dart codebase for iOS devices, integrating Apple Push Notification service (APNs), Sign in with Apple, and optimizing the Flutter rendering engine for iOS hardware.



Questions?





THANK YOU!